Optimizing Surgical Patient Flow through Operating Room Scheduling: A Use Case Study

Walid Abdelaidoum [1,2], Mohamed A. Madani [2], Larbi Boubchir [1], Boubaker Daachi [1], Louis Dagneaux [3] and Stanislas de Coüasnon[2]
[1] *LIASD, University of Paris 8, Paris, France*
[2] *SWAPPY, Paris, France*
[3] *Hôpital Lapeyronie, Orthopedic Service, Montpellier, France*
walid.abdelaidoum@etud.univ-paris8.fr, abdou@swappy.fr, larbi.boubchir@univ-paris8.fr,
professeur.dagneaux@gmail.com, sdecouasnon@swappy.fr

***Abstract*—Operating room (OR) scheduling often prioritizes operational efficiency over patient preferences. In our study, we present a patient-centered scheduling framework developed for the orthopedic department at CHU Montpellier (France). Formulated as a mixed-integer linear program (MILP), the model handles strict resource constraints with patient availabilities. This framework generates a set of optimized time slots, letting patients select their preferred surgery date. We used an iterative heuristic that proposes a patient at a time, where the physician sets with the patient the best schedule date between three options, allowing the excluded ones to be considered for the other patients. The results using use case data show that the MILP model increases the theoretical capacity of the operating rooms utilization by 23%. Benchmarks were also generated to simulate daily uses and showed that, on average, more than 2.5 slots are proposed to each patient. The results show that integrating patient preferences improves utilization and patient satisfaction.**

***Index Terms*—Operating Room Scheduling, Surgical Patient Flow, Mixed-integer Linear Programming, Optimization, eHealth.**

I. Introduction

Operating room scheduling is a multi-objective optimization problem [2]. The objectives are to minimize costs, for instance, surgical care accounts for nearly one third of US health care spending [4], and to maximize operational utilization under hospital constraints, while accounting for staff and patient needs. Most studies propose deterministic or stochastic models that generate a predetermined schedule without patient choice [8, 18, 20]. This is reasonable for time-critical cases such as cardiovascular or emergency surgery.

However, this study is set in the orthopedic department at the Centre Hospitalier Universitaire (CHU) Montpellier (France), where a patient-centered scheduling approach is practical. Indeed, unlike emergency or time-critical surgical cases, elective orthopedic patients generally have more flexibility in their scheduling preferences, even when urgent cases are involved. This flexibility allows patient preferences to be included in scheduling. This work balances operational efficiency with patient care by letting patients participate in scheduling.

As shown in Figure 1, this approach is based on an iterative optimization methodology. First, patient and physician data are uploaded into the system. Based on this dataset, the MILP model generates up to three slots that match the availability of both patients and physicians. Next, the flexibility-first heuristic identifies the patient with the most flexible schedule,

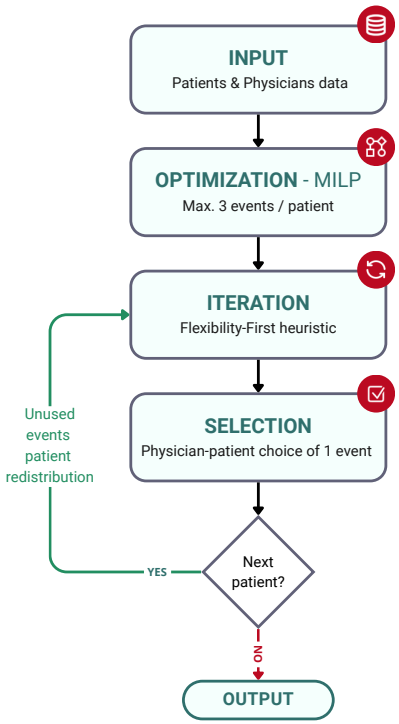

FIG. 1: Iterative scheduling process: MILP generates feasible events, the flexibility-first heuristic selects and confirms patient selections, and unused events are redistributed.

as determined by the quality scores σ_p (detailed in Subsection III-D). The selected patient reviews the available options and chooses the one that works best for them. Then, the unused options are redistributed to the remaining patients, and the system recalculates which patient to process next until there are no patients left to assign. This iterative method includes patients in the scheduling loop while preserving efficiency.

II. Related Work

Operating room scheduling has been widely studied [2]. Among heuristic approaches, Kamran et al. [8] applied a column-generation-heuristic-based Benders' decomposition on adaptive allocation scheduling of patients in operating rooms, while Wang et al. [15] developed a Wasserstein distributionally robust optimization model with a rolling horizon rescheduling scheme for surgery scheduling involving both elective and emergency patients. AI-based methods

have also been used. Lex et al. [9] combined machine learning prediction with scheduling optimization for elective orthopedic surgery. Zhao et al. [19] integrated reinforcement learning with column generation for OR scheduling. El-Balka et al. [6] proposed a dynamic scheduling strategy combining explainable AI with a fuzzy engine for intraday scheduling. Eshghali et al. [7] used random forest with genetic algorithm and particle swarm optimization for elective and emergency patient scheduling. Stochastic approaches address uncertainties in surgical scheduling [3]. M'Hallah and Visintin [11] applied stochastic integer programming for the Master Surgical Schedule. Zhang et al. [18] combined Markov decision processes with stochastic programming for surgery scheduling. Among exact methods, Çelik et al. [20] used a two-stage stochastic MIP with progressive hedging. Schneider et al. [14] employed a MILP for surgery group scheduling with downstream resources. Miao and Wang [12] proposed a MIP for elective and emergency scheduling with waiting time limits. Akbarzadeh and Maenhout [1] developed a branch-price-and-cut algorithm for patient and surgeon scheduling. Makboul et al. [10] applied multiobjective ϵ-constraint robust optimization for the master surgical schedule. Xue et al. [16] addressed multi-resource constrained elective surgical scheduling using Nash equilibrium, while [5] applied distributionally robust optimization to integrated inpatient and ambulatory surgery scheduling.

Closest to our work, Spratt and Kozan [13] proposed constructive heuristics with real-time reactive strategies for surgical case sequencing across multiple operating rooms.

Yang et al. [17] recently integrated patient satisfaction into OR scheduling using a multinomial logit model to capture surgeon and date preferences. Our work differs by integrating patient choice directly into the scheduling process: the model proposes two to three feasible slots per patient and guarantees at least one option for every patient.

III. MILP Model with Patient Preference

A. Operational Context

Unlike batch scheduling, the system processes patients incrementally after consultation. The physician determines surgical need and adds the patient to the schedule. The optimization model then generates up to three feasible time slots per patient. Patients review these options and confirm their preferred date; confirmed assignments become fixed for subsequent runs. Each physician's wait list typically contains 5–15 patients pending slot generation, patient confirmation, or prerequisite resolution.

B. Constraints description

We formulate the scheduling decision as an optimization problem satisfying multiple constraints and optimizing a hierarchical multi-objective function (data notations are in Table I):

Clinical and Temporal Constraints: The solution must respect essential patient-specific requirements, including the operating time (τ_p), post-operative time (τ_p^{post}), and the patient's earliest (d_p^{arr}) and latest (d_p^{dep}) feasible scheduling days. Furthermore, the schedule must adhere to the resource availability (OR opening $B_{b,d}$, capacity $C_{b,d}$) and the compatibility between patients and physicians ($w_{p,h}$).

Hierarchical Objectives: The objective minimizes a weighted sum of violations, structured hierarchically by penalty weights (α_1 to α_5):

- **Priority 1:** Minimize unassigned patients (α_1).
- **Priority 2:** Minimize capacity and time constraint violations, including early or late scheduling relative to the patient's stay limits (α_3). Capacity violations (α_2) are permitted only for urgent cases replacing elective patients.
- **Priority 3:** Minimize specialty fragmentation (α_4) and efficiently manage day usage (α_5), to reduce specialty fragmentation and concentrate day usage.

C. Mathematical model

The operational scheduling problem is formulated as a MILP, with the objective to assign patients across operating rooms and days. The model's sets and parameters are detailed in Table I, and its decision variables are defined in Table II.

1) Objective: The objective function maximizes Z using hierarchically ordered weights (α_k). Patient assignment receives highest priority: the model minimizes assignment errors ($\alpha_1 = 10^3$, $e_{p,i}$) while maximizing rewards for high-priority patients (π_p), preferred physician matches ($w_{p,h}$), and favorable block-day assignments ($a_{p,h,b,d}$). Earlier intra-day scheduling (smaller index i) receives proportionally higher reward.

$$\max\ Z = + \sum_{d,b,h,p,i} X_{d,b,h,p,i} \Big[\pi_p(T-d) + w_{p,h} + a_{p,h,b,d}\Big](i+1)^{-1} \tag{1}$$

$$- \alpha_1 \sum_{p,i} \pi_p\,(i+1)^{-5}\, e_{p,i} \tag{2}$$

$$- \alpha_2 \sum_{b,d} E_{b,d} \tag{3}$$

$$- \alpha_3 \sum_{p,i} (EA_{p,i} + LD_{p,i}) \tag{4}$$

$$- \alpha_4 \sum_{b,h} N_{b,h} \tag{5}$$

$$- \alpha_5 \sum_{d} d\, z_d \tag{6}$$

2) Constraints: **Patient one-time assignment (with slack).**

$$\sum_{d\in\mathcal{D}} \sum_{b\in\mathcal{B}} \sum_{h\in\mathcal{H}} X_{d,b,h,p,i} + e_{p,i} = 1, \qquad \forall p \in \mathcal{P},\ \forall i \in \mathcal{I} \tag{7}$$

(If $e_{p,i} = 1$ then p is not assigned at index i; otherwise p must be assigned exactly once at that index.)

Daily block capacity constraints (with slack).

$$\sum_{h\in\mathcal{H}} \sum_{p\in\mathcal{P}} \sum_{i\in\mathcal{I}} X_{d,b,h,p,i}\,(\tau_p + \tau_p^{\text{post}}) \le C_{b,d} + \text{stdPost} + E_{b,d} \tag{8}$$

$$0 \le E_{b,d} \le \overline{E}_{b,d}, \qquad \forall b \in \mathcal{B},\ \forall d \in \mathcal{D} : B_{b,d} = 1 \tag{9}$$

Patient availability vs. arrival/departure (early/late violations).

TABLE I: Data description (sets and parameters).

Symbol	Description
$\mathcal{D}$	Set of planning days, index d
$\mathcal{B}$	Set of operating rooms (blocs), index b
$\mathcal{H}$	Set of physicians, index h
$\mathcal{P}$	Set of patients, index p
$\mathcal{I} = \{0, \ldots, I_{\max} - 1\}$	Set of proposed time slot option indices for each patient, index i
$\mathcal{K}$	Set of operation types, index k
$\mathcal{D}^{\text{part}}$	Family of day partitions (for spacing constraint), each partition $P \in \mathcal{D}^{\text{part}}$ is a subset of $\mathcal{D}$
τ_p	Operating time of patient p
τ_p^{post}	Post-operative time of patient p
π_p	Priority of patient p
$k(p) \in \mathcal{K}$	Operation type of patient p
d_p^{arr}	Earliest feasible day index for patient p
d_p^{dep}	Latest feasible day index for patient p
T	Planning horizon length (in days); $\mathcal{D} = \{0, \ldots, T-1\}$
stdPost	Standard post-operative buffer time (scheduling flexibility allowance added to block capacity)
$B_{b,d} \in \{0,1\}$	Indicator: block b is open on day d (when $B_{b,d} = 0$, capacity is zero)
$C_{b,d}$	Available OR-time on (b, d)
$\overline{E}_{b,d}$	Upper bound on capacity violation slack
$w_{p,h}$	Compatibility weight between patient p and physician h
$a_{p,h,b,d}$	Preference weight for assigning (p, h) to (b, d)
$n_{b,d}^{\text{pre}}$	Number of pre-defined operation types in block b on day d (aggregated across all physicians)
M	Big-M constant, e.g. $M \geq \lvert\mathcal{B}\rvert \cdot \lvert\mathcal{H}\rvert \cdot \lvert\mathcal{P}\rvert \cdot \lvert\mathcal{I}\rvert$
$\alpha_1 = 10^3$	Weight of patient-assignment errors
$\alpha_2 = 10^1$	The weight of the block capacity violation (is only used to schedule urgency, replacing elective patients).
$\alpha_3 = 10^1$	Weight of early/late scheduling w.r.t. stay
$\alpha_4 = 10^{-1}$	Weight of specialty fragmentation
$\alpha_5 = 10^{-3}$	Weight of day usage

TABLE II: Decision variables.

Variable	Domain	Description
$X_{d,b,h,p,i}$	$\{0,1\}$	1 if patient p is scheduled on day d in block b with physician h and index i
z_d	$\{0,1\}$	1 if day d is used (at least one assignment), 0 otherwise
$e_{p,i}$	$\{0,1\}$	1 if patient p is not assigned at index i (slack for one-time assignment)
$E_{b,d}$	$\mathbb{Z}_{\geq 0}$	Capacity violation (extra time) allowed for block b on day d
$EA_{p,i}$	$\{0,1\}$	Early-arrival violation indicator for patient p at index i
$LD_{p,i}$	$\{0,1\}$	Late-departure violation indicator for patient p at index i
$N_{b,h}$	$\mathbb{Z}_{\geq 0}$	Upper bound on the number of distinct operation types for physician h in block b on any single day
$u_{d,b,h,k}$	$\{0,1\}$	1 if physician h performs operation type k in block b on day d

Let d_p^{arr} and d_p^{dep} be the first/last admissible day indices for patient p.

$$\sum_{\substack{d \in \mathcal{D}: \\ d < d_p^{\text{arr}}}} \sum_{b \in \mathcal{B}} \sum_{h \in \mathcal{H}} X_{d,b,h,p,i} = EA_{p,i}, \quad \forall p \in \mathcal{P},\ \forall i \in \mathcal{I} \tag{10}$$

($EA_{p,i}$ may be fixed to 0 if early reprogramming is not allowed for p).

$$\sum_{\substack{d \in \mathcal{D}: \\ d > d_p^{\text{dep}}}} \sum_{b \in \mathcal{B}} \sum_{h \in \mathcal{H}} X_{d,b,h,p,i} = LD_{p,i}, \quad \forall p \in \mathcal{P},\ \forall i \in \mathcal{I} \tag{11}$$

($LD_{p,i}$ is fixed to 0 if late reprogramming is not allowed for p).

Day-activation constraints.

$$\sum_{b \in \mathcal{B}} \sum_{h \in \mathcal{H}} \sum_{p \in \mathcal{P}} \sum_{i \in \mathcal{I}} X_{d,b,h,p,i} \leq M \cdot z_d, \qquad \forall d \in \mathcal{D} \tag{12}$$

If any $X_{d,b,h,p,i} = 1$ for a given d, then z_d must be 1 (and the day is penalized in the objective by (6)).

Specialty indicators and fragmentation per doctor and room.

$$u_{d,b,h,k} \geq X_{d,b,h,p,i}, \qquad \forall p \in \mathcal{P} : k(p) = k, \quad \forall i \in \mathcal{I} \tag{13}$$

$$u_{d,b,h,k} \leq \sum_{p \in \mathcal{P}: k(p) = k} \sum_{i \in \mathcal{I}} X_{d,b,h,p,i} \tag{14}$$

so $u_{d,b,h,k}$ becomes 1 if (b, h) actually executes at least one case of type k on day d.

The aggregated number of specialties per physician and block is constrained by

$$\sum_{k \in \mathcal{K}} u_{d,b,h,k} - 1 + n_{b,d}^{\text{pre}} \leq N_{b,h}, \quad \forall d \in \mathcal{D}, \forall b \in \mathcal{B}, \forall h \in \mathcal{H} \tag{15}$$

Minimum spacing between slots per patient

$$\sum_{d \in P} \sum_{b \in \mathcal{B}} \sum_{i \in \mathcal{I}} X_{d,b,h,p,i} \leq 1, \quad \forall P \in \mathcal{D}^{\text{part}}, \forall h \in \mathcal{H}, \forall p \in \mathcal{P} \tag{16}$$

With partition interval equal to 1 in the code, this effectively prevents multiple appointments of the same (p, h) within the same partition interval.

D. Flexibility-First Heuristic

Patients select their surgery time slots sequentially rather than simultaneously. Once a patient confirms a time slot, it becomes fixed, and subsequent optimizations will not take it into consideration for further recalculations. This reduces the possible options for patients remaining in the pool. Therefore, the order in which patients are presented for selection is critical to maximizing the feasibility of the solution and preserving the best options for all patients. Unlike standard greedy or priority-based sequencing methods, this heuristic re-solves the MILP after each confirmation, updating available slots for all remaining patients rather than assigning from a static list.

This heuristic prioritizes patients with the most flexible schedules (high availability scores) based on the MILP results for early selection. These patients can choose from multiple feasible slots. For each unscheduled patient p, an availability score σ_p quantifies scheduling flexibility:

$$\sigma_p = \sum_{s \in \mathcal{S}_p} \text{score}(s, p) \tag{17}$$

where $\mathcal{S}_p$ is the set of feasible slots and $\text{score}(s, p)$ is:

$$\text{score}(s, p) = \begin{cases} +100 & \text{if } s \in [d_p^{\text{arr}}, d_p^{\text{dep}}] \\ -|s - d_p^{\text{arr}}| & \text{if } s < d_p^{\text{arr}} \\ -|s - d_p^{\text{dep}}| & \text{if } s > d_p^{\text{dep}} \end{cases} \tag{18}$$

Patients with higher availability scores σ_p are prioritized in the selection sequence. After each patient confirms a selection, slot availability is updated and scores are recomputed for remaining patients, as detailed in Algorithm 1. For each patient p, the set of proposed slots $\mathcal{S}_p$ is derived from the MILP solution: each slot corresponds to a (d, b, h) triple where $X_{d,b,h,p,i} = 1$. Confirming a slot $s^* \in \mathcal{S}_{p^*}$ fixes the corresponding X variables in subsequent MILP re-optimizations.

At each iteration, the re-optimization step validates remaining slots against updated constraints: the assigned physician must be available at the candidate block and day, the patient must not already hold a confirmed slot on that day, the scheduling day must fall within the patient's feasibility window, and the residual block capacity must accommodate the intervention. Block capacity is decremented by $(\tau_p + \tau_p^{\text{post}})$ after each confirmed assignment, so that feasibility checks reflect current resource usage. Each patient carries a reprogramming type—*before*, *after*, or *between*—that determines whether scheduling outside $[d_p^{\text{arr}}, d_p^{\text{dep}}]$ is permitted, restricting the set $\mathcal{S}_p$ accordingly. When a selected slot shares its block-day with another patient's proposed slot, the system detects the conflict and flags the displaced patient. Finally, confirmed assignments are irreversible: the heuristic does not revisit previous decisions, bounding computation at the cost of global optimality.

Algorithm 1 Iterative Patient Selection and Confirmation

Input: Proposed slots $\mathcal{S}_p$ for each patient $p \in \mathcal{P}$, current assignments $A \leftarrow \emptyset$
Output: Confirmed assignments A^*
Initialize $A^* \leftarrow \emptyset$, $\mathcal{P}^{\text{remain}} \leftarrow \mathcal{P}$
while $\mathcal{P}^{\text{remain}} \neq \emptyset$ **do**
  for each patient $p \in \mathcal{P}^{\text{remain}}$ **do**
    Compute score: $\sigma_p \leftarrow \sum_{s \in \mathcal{S}_p} \text{score}(s, p)$
  end for
  $p^* \leftarrow \arg\max_p \sigma_p$ (select most flexible patient)
  Sort $\mathcal{S}_{p^*}$ by $\text{score}(s, p^*)$ in descending order
  Display sorted slots to user (physician or patient)
  Receive user confirmation: $s^* \in \mathcal{S}_{p^*}$
  $A^* \leftarrow A^* \cup \{(p^*, s^*)\}$
  $\mathcal{P}^{\text{remain}} \leftarrow \mathcal{P}^{\text{remain}} \setminus \{p^*\}$
  Re-optimize: Update $\mathcal{S}_p$ for all $p \in \mathcal{P}^{\text{remain}}$ given fixed assignments A^*
end while
return A^*

IV. Numerical Results

A. Hospital Data Structure

The computational experiments use data from the Department of Orthopedic Surgery at CHU Montpellier for elective surgical procedures over a three-week period. This framework is used to test the model's ability to optimize and condense the pre-existing surgical schedule.

The dataset includes 134 patients requiring surgery, categorized into four main procedure types: arthroplasty, foot and ankle surgery, soft tissue repair, and arthroscopy.

Operative time ranges from 15 to 240 minutes, with mean duration 71.86 minutes and median 80 minutes. A fixed 50-minute post-operative turnover time is assumed for all cases.

A total of 11 physicians are involved. Their schedules are fixed, requiring availability in at least one of the two weeks modeled. Two head physicians are included; one is capable of managing "swinging rooms," operating concurrently in up to 3 Operating Rooms (ORs).

Three OR blocks are available all working weekdays, providing 10.5 hours of capacity daily. One limited OR is open only on Wednesdays and Fridays, providing 6.5 hours of capacity daily.

B. Real-World Performance

The baseline ("Real Case") schedule was produced manually by administrative staff at CHU Montpellier using conventional, non-optimized procedures. The results, summarized in Table III and Figure 2, show that the model improves resource utilization while maintaining scheduling flexibility. To isolate maximum resource utilization, tests limited each patient to a single scheduled event. This step directly assessed the theoretical capacity within existing real-world resource constraints.

The model redistributes patients across the planning horizon. In Weeks 1–2, utilization increased from 72–73% to 91–95% by condensing workload into available block time. In the third week, utilization decreased from 79% to 39% because most patients had already been assigned to the first two weeks, effectively freeing Week 3 capacity for future demand. In

TABLE III: Comparison of Surgical Block Time Utilization (%): Real Case vs. Optimized Schedule over 3 Weeks

Week	Real Case (%)	Optimized (%)	Change (p.p.)
1	72%	95%	+23
2	73%	91%	+18
3	79%	39%	-40

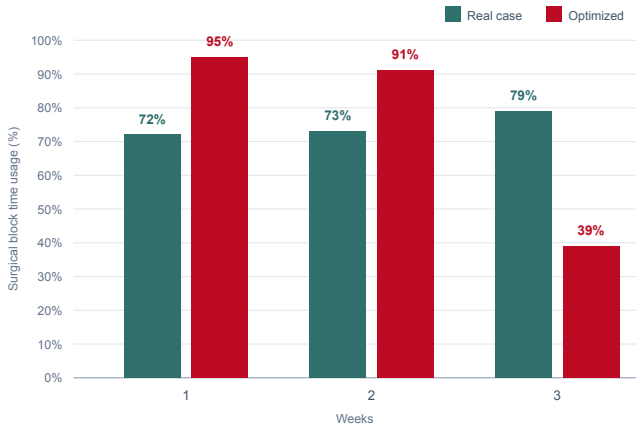

FIG. 2: Surgical Block Time Utilization: Real Case vs. Optimized Schedule.

these tests, we did not include fictitious patients; the objective was to redistribute existing patients to evaluate whether utilization gains were achievable in previously completed schedules.

C. Simulation Results

Benchmark instances of 10–40 patients match realistic operational scales, given the continuous-flow model described in Section III-A (Table IV).

TABLE IV: Benchmark Data Results (mean values over 20 independent runs per configuration)

Physicians	Patients	Blocs	Events	Patients with Events	Avg per Patient	Run Time
1	10	2	30	10	3.0	0.17
1	10	3	30	10	3.0	0.25
2	10	2	30	10	3.0	0.34
2	10	3	30	10	3.0	0.45
1	20	2	60	20	3.0	0.98
1	20	3	60	20	3.0	1.89
2	20	2	60	20	3.0	3.40
2	20	3	60	20	3.0	3.94
1	40	2	98.34	40	2.74	600.0
1	40	3	111.15	40	2.77	600.0
2	40	2	100.53	40	2.50	600.0
2	40	3	111.56	40	2.77	600.0

The model was evaluated across three dimensions: patient count ($P \in \{10, 20, 40\}$), number of physicians ($\{1, 2\}$), and operating room blocks ($\{2, 3\}$). For each configuration, up to three feasible time slots per patient were generated, yielding 30–111 total candidate events. To assess computational robustness, each configuration was solved 20 times with different random seeds for incoming patient characteristics while maintaining the baseline schedule structure; all reported metrics (events, average options per patient, runtime) represent means across these 20 replications (Table IV).

Key findings: (1) For operational scales ($P \leq 20$), mean solution times remain below 4 seconds across all configurations, enabling real-time consultation-integrated workflow (Figure 4). (2) For larger instances ($P = 40$) reaching the 600-second timeout, solutions remain feasible with mean optimality gaps below 15% and sufficient option generation (2.50–2.77 options per patient; Figure 3). These batch scenarios are infrequent in practice due to the continuous-flow model; 600 seconds is acceptable for offline processing. Moreover, the optimality gap affects only the ranking of proposed slots, not the feasibility of the schedule: patients still receive at least 2.50 options on average. (3) The 20-run replication design confirms computational stability: for $P \leq 20$, optimal solutions are consistently achieved within seconds.

The flexibility-first heuristic (Algorithm 1) then guides sequential patient confirmation, preserving options for constrained patients.

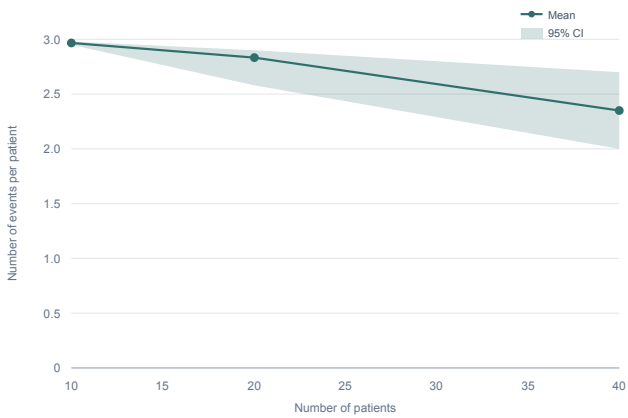

FIG. 3: Average slots per scheduled patient

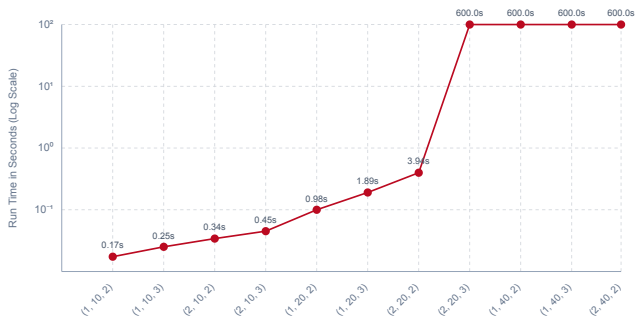

FIG. 4: Run time per instance (line plot with logarithmic scale)

V. CONCLUSION

This patient-centered OR scheduling framework combines operational optimization with patient choice. Implemented in the orthopedic department at CHU Montpellier (France), the system is computationally feasible and operationally useful for patient-centered surgical scheduling.

The solution uses a MILP with hierarchical objectives to generate up to three time slot options for each patient. A flexibility-first heuristic prioritizes patients with flexible schedules based on early selection and availability scores

while preserving options for those with constrained schedules. Patients actively participate in their scheduling decision by selecting their preferred slot option.

Benchmark simulations across 10–40 patients, 1–2 physicians, and 2–3 operating rooms cover a range of realistic scenarios. Real-world validation (134 patients, 3 weeks) achieved 72–73% to 91–95% utilization increases and 36 net operations gained versus 11 in the real case. For scales $P \leq 20$, solutions arrive in under 4 seconds, enabling real-time workflow. The weighted hierarchical objective balances OR capacity with shared decision-making.

The framework was developed for elective orthopedic surgery, where scheduling flexibility is higher. Validation in other surgical specialties and at larger institutional scales is necessary.

Future work includes modeling stochastic durations, learning patient preferences, and real-time rescheduling. Genetic algorithms could improve scalability for larger instances.

Patient preferences can be incorporated into OR scheduling without prohibitive computational cost.

References

[1] Babak Akbarzadeh and Broos Maenhout. A dedicated branch-price-and-cut algorithm for advance patient planning and surgeon scheduling. *Eur. J. Oper. Res.*, 322(2):448–466, 2025.

[2] Md Al Amin et al. A comprehensive review on operating room scheduling and optimization. *Oper. Res.*, 25(1):3, 2025.

[3] Ambrogio Maria Bernardelli et al. Multi-objective stochastic scheduling of inpatient and outpatient surgeries. *Flex. Serv. Manuf. J.*, 37:1148–1202, 2025.

[4] Christopher P. Childers and Melinda Maggard-Gibbons. Understanding costs of care in the operating room. *JAMA Surgery*, 153(4):e176233–e176233, 04 2018.

[5] Zongli Dai et al. Integrated robust scheduling for inpatient surgery and ambulatory surgery. *Comput. Oper. Res.*, page 107098, 2025.

[6] R. M. El-Balka, N. Sakr, A. H. Rabie, et al. A dynamic operation room scheduling strategy based on explainable ai and fuzzy interface engine. *Artif. Intell. Rev.*, 58:365, 2025.

[7] Masoud Eshghali et al. Machine learning based integrated scheduling and rescheduling for elective and emergency patients in the operating theatre. *Ann. Oper. Res.*, 332:989–1012, 2024.

[8] Mehdi A. Kamran et al. A column-generation-heuristic-based benders' decomposition for solving adaptive allocation scheduling of patients in operating rooms. *Comput. Ind. Eng.*, 148:106698, 2020.

[9] Johnathan R. Lex et al. Using machine learning to predict-then-optimize elective orthopedic surgery scheduling to improve operating room utilization: Retrospective study. *JMIR Med. Inform.*, 13:e70857, 2025.

[10] Salma Makboul et al. A multiobjective ϵ-constraint based approach for the robust master surgical schedule under multiple uncertainties. *Eur. J. Oper. Res.*, 320(3):682–698, 2025.

[11] Rym M'Hallah and Filippo Visintin. A stochastic model for scheduling elective surgeries in a cyclic master surgical schedule. *Comput. Ind. Eng.*, 129:156–168, 2019.

[12] Hongru Miao and Jian-Jun Wang. Scheduling elective and emergency surgeries at shared operating rooms with emergency uncertainty and waiting time limit. *Comput. Ind. Eng.*, 160:107551, 2021.

[13] Belinda Spratt and Erhan Kozan. A real-time reactive framework for the surgical case sequencing problem. *Flex. Serv. Manuf. J.*, 33:183–211, 3 2021.

[14] A.J. Thomas Schneider et al. Scheduling surgery groups considering multiple downstream resources. *Eur. J. Oper. Res.*, 282(2):741–752, 2020.

[15] Yu Wang et al. Wasserstein distributionally robust surgery scheduling with elective and emergency patients. *Eur. J. Oper. Res.*, 314(2):509–522, 2024.

[16] Jun Xue et al. Multi-resource constrained elective surgical scheduling with nash equilibrium toward smart hospitals. *Sci. Rep.*, 15(1):3946, 2025.

[17] Xue Yang, Haichao Liu, Yang Wang, Abraham P. Punnen, and Haibo Wang. Integrating patient satisfaction into operating room scheduling decisions. *Comput. Ind. Eng.*, 2025.

[18] Jian Zhang et al. A two-phase optimization model combining markov decision process and stochastic programming for advance surgery scheduling. *Comput. Ind. Eng.*, 160:107548, 2021.

[19] R. Zhao et al. Optimizing operating room scheduling through multi-level learning and column generation: a novel hybrid approach. *Health Care Manag. Sci.*, 28, 2025.

[20] Batuhan Çelik et al. A stochastic programming approach to surgery scheduling under parallel processing principle. *Omega*, 115:102799, 2023.